MANUAL PASO A PASO CON TDD

Dolores Hoy

Flutter + 5 APIs gratuitas con buenas prácticas de seguridad

Clima · Tipo de cambio · Festivos · Wikipedia · NASA

Desarrollo Móvil Integral · UTNG

Prof. Anastacio Rodríguez García

Antes de empezar: ¿qué vamos a construir?

Imagina el tablero de información de la plaza de Dolores Hidalgo: te dice qué clima hace, a cuánto está el dólar, cuándo es el próximo día festivo, algo de historia del lugar y hasta la foto astronómica del día. Cinco datos, cinco fuentes distintas. Ninguna es nuestra: las "pedimos prestadas" a servicios de internet gratuitos llamados APIs.

Las 5 APIs del proyecto

- 1 · Open-Meteo: clima actual de Dolores Hidalgo. Sin clave.
- 2 · Frankfurter: tipo de cambio peso-dólar-euro (datos del Banco Central Europeo). Sin clave.
- 3 · Nager.Date: días festivos oficiales de México. Sin clave.
- 4 · Wikipedia en español: resumen de un tema escrito por el usuario. Sin clave, pero recibe TEXTO del usuario.
- 5 · NASA APOD: foto astronómica del día. REQUIERE CLAVE (API key). Es la API que usaremos para estudiar seguridad a fondo.

> **Dato curioso / ejemplo**
> Las claves de API son como la llave de tu casa: se la das a la app para entrar a un servicio. Si la llave cae en manos equivocadas, otra persona puede usar tu cuenta, gastar tu cupo o, en APIs de pago, cobrarte dinero. En 2023 se reportaron cientos de casos de claves publicadas por accidente en GitHub: los bots las encuentran en segundos.

Lo que aprenderás

- Consumir 5 APIs distintas con un mismo cliente HTTP reutilizable.
- Probar cada API con MockClient, sin internet y sin gastar cupo.
- Seguridad aplicada: HTTPS obligatorio, claves fuera del código, validación de entradas y salidas, límites de tamaño y de tiempo, mensajes de error que no filtran información, logs sin secretos.
- Hacer que una API caída NO tumbe a las otras cuatro.

Las dos ideas de seguridad que guían todo el manual

1. Nunca confíes en lo que llega de internet (respuestas del servidor) ni en lo que escribe el usuario: se valida SIEMPRE.
2. Nunca escribas un secreto donde pueda leerlo alguien más: ni en el código, ni en el repositorio, ni en los logs, ni en los mensajes de error.

Mapa del proyecto

```
dolores_hoy/
  lib/
    dominio/
      excepciones.dart    <- errores con mensajes seguros
      modelos.dart        <- Clima, TipoCambio, Festivo, ResumenWiki, FotoAstronomica
    datos/
      cliente_seguro.dart <- ÚNICA puerta a internet (https, timeout, límites)
      config_claves.dart  <- claves que entran al compilar, nunca en el código
      servicios.dart      <- una clase por API
    presentacion/
```

```
    tablero_viewmodel.dart   pantalla_tablero.dart
  main.dart
test/
  ayudas.dart  cliente_seguro_test.dart  servicios_test.dart
  seguridad_test.dart  tablero_viewmodel_test.dart
secrets.json            <- TU clave (NO se sube a git)
secrets.json.example    <- plantilla sin secretos (SÍ se sube)
```

El ciclo TDD de siempre

ROJO: escribe la prueba (debe FALLAR)
Escribes la prueba; falla.

VERDE: el código mínimo para pasar
Código mínimo para que pase.

REFACTOR: ordena sin romper nada
Ordenas sin romper nada.

⚠ Cuidado
Las respuestas simuladas de las pruebas están basadas en la documentación de cada API. Si algún día una API cambia su formato, tu app dará "datos que no entiendo" en lugar de tronar; entonces actualizas el modelo y la prueba. Antes de la clase abre cada URL en el navegador para confirmar que el formato sigue igual.

Ronda 0 · Preparar el taller

Paso 0.1 · Crear el proyecto

```
$ flutter create dolores_hoy
$ cd dolores_hoy
$ mkdir lib/dominio lib/datos lib/presentacion
```

Paso 0.2 · Dependencias

Solo necesitamos el paquete http. Reemplaza pubspec.yaml y borra test/widget_test.dart:

pubspec.yaml

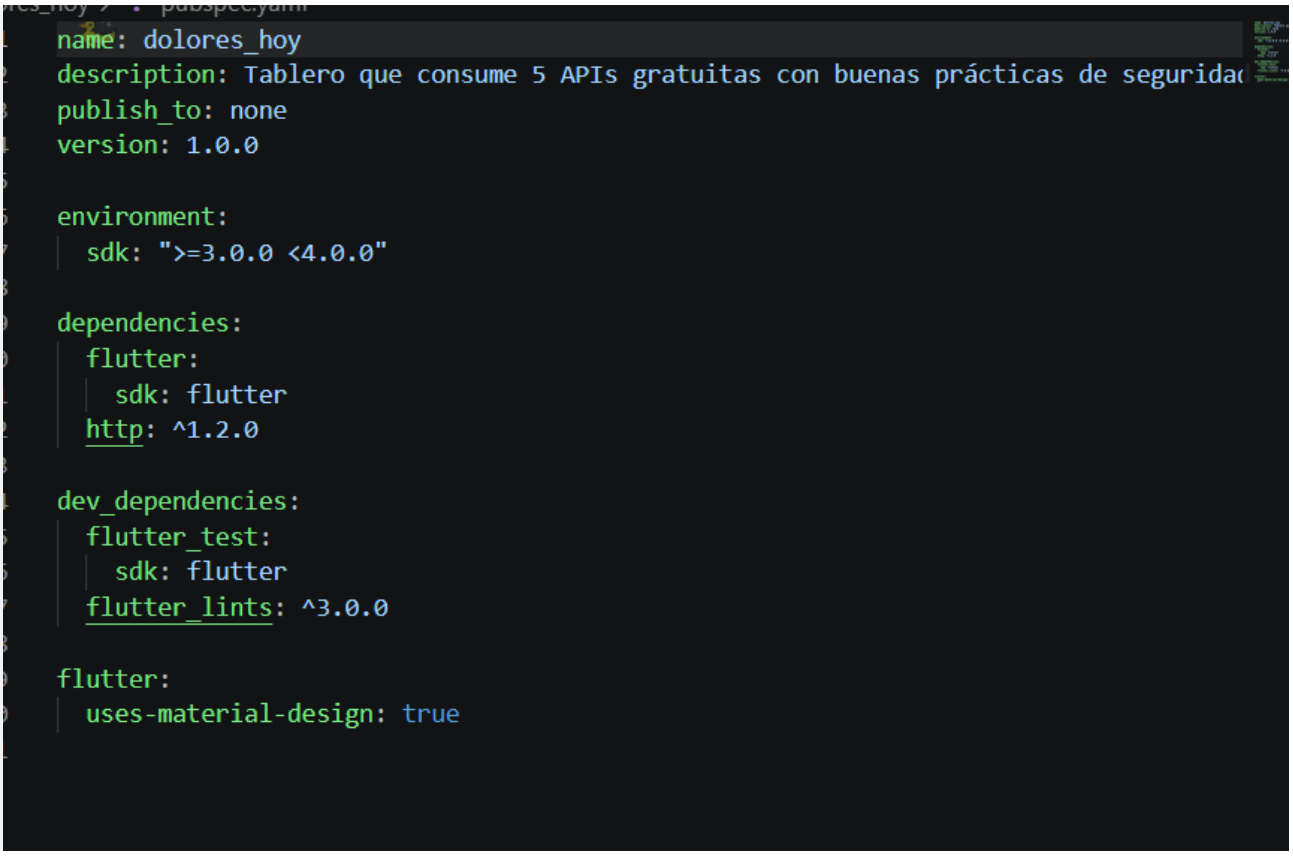
```
name: dolores_hoy
description: Tablero que consume 5 APIs gratuitas con buenas prácticas de seguridad.
publish_to: none
version: 1.0.0

environment:
  sdk: ">=3.0.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true.
```



```yaml
name: dolores_hoy
description: Tablero que consume 5 APIs gratuitas con buenas prácticas de seguridad
publish_to: none
version: 1.0.0

environment:
  sdk: ">=3.0.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
```

```
$ flutter pub get
```

Paso 0.3 · Permiso de internet en Android

En los builds de depuración ya viene incluido, pero para la versión final verifica que android/app/src/main/AndroidManifest.xml tenga:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Observa que NO activamos usesCleartextTraffic como en el proyecto de API local: aquí TODAS las APIs usan https y queremos que Android nos proteja si algún día alguien escribe una dirección http por error.

Paso 0.4 · Preparar los archivos de secretos (hazlo desde el inicio)

Antes de escribir una sola línea de código, protege el repositorio. Crea .gitignore con estas líneas (agrégalas a las que Flutter ya generó):

.gitignore (agregar al final)

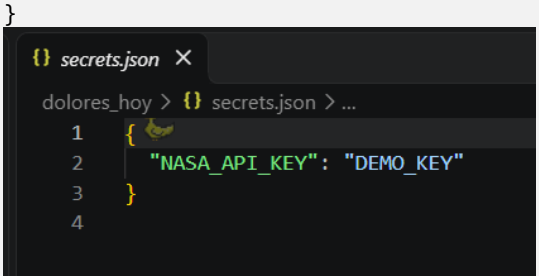
```
# Secretos: JAMAS al repositorio
secrets.json
*.env
.env*
*.jks
*.keystore
key.properties
```

```
secrets.json
*.env
.env*
*.jks
*.keystore
key.properties
```

Crea el archivo secrets.json.example, que SÍ se sube al repositorio como plantilla para tus compañeros:

secrets.json.example

```
{
  "NASA_API_KEY": "DEMO_KEY"
}
```



Y copia ese archivo con el nombre secrets.json (el que SÍ contiene tu clave real y que git ignorará). Por ahora deja DEMO_KEY.

```
$ cp secrets.json.example secrets.json
$ git status
```

> **Dato curioso / ejemplo**
> Comprueba con git status que secrets.json NO aparece en la lista de archivos por subir. Si aparece, tu .gitignore está mal escrito. Hacer esta revisión una vez te evita el peor día de tu carrera: el de subir tus claves a GitHub.

> **⚠ Cuidado**
> Si algún día subes una clave por accidente, borrar el archivo NO basta: queda en el historial de git. Lo correcto es revocar la clave inmediatamente y generar una nueva.

Ronda 1 · Conociendo las 5 APIs

Antes de programar, lee la documentación

Una API gratuita no significa "sin reglas". Cada una tiene límites de uso y términos que debes respetar. Es como el agua de la fuente de la plaza: es gratis, pero si alguien se lleva toda en una pipa, deja sin agua a los demás.

Ficha técnica de cada API

- Open-Meteo · https://api.open-meteo.com/v1/forecast · parámetros: latitude, longitude, current, timezone · sin clave · uso no comercial gratuito con límite diario generoso.
- Frankfurter · https://api.frankfurter.dev/v1/latest · parámetros: base, symbols · sin clave · datos del Banco Central Europeo, se actualizan en días hábiles.
- Nager.Date · https://date.nager.at/api/v3/PublicHolidays/AÑO/MX · sin clave · festivos oficiales por país.
- Wikipedia · https://es.wikipedia.org/api/rest_v1/page/summary/TEMA · sin clave · pide identificarte con un User-Agent descriptivo (lo haremos en el cliente).
- NASA APOD · https://api.nasa.gov/planetary/apod?api_key=TU_CLAVE · clave gratuita en api.nasa.gov · con DEMO_KEY el límite es muy bajo (pocas peticiones por hora por dirección IP); con tu clave personal, mucho mayor.

Anatomía de una petición

Toda dirección tiene partes. Tómate un minuto para identificarlas en esta:

```
https://api.nasa.gov/planetary/apod?api_key=ABC123&date=2026-10-29
  ↑esquema  ↑host       ↑ruta      ↑ parámetros (clave=valor, separados por &)
```

- Esquema: https cifra la conversación; http NO (cualquiera en la misma red Wi-Fi puede leerla).
- Host: el servidor al que llamas.
- Ruta: qué recurso pides.
- Parámetros: opciones; aquí viaja la clave. ¡Por eso las URLs con clave no deben ir a los logs!

> **Dato curioso / ejemplo**
> En una red Wi-Fi pública (cafés, aeropuertos), alguien con una herramienta gratuita puede ver todo el tráfico http de los demás. Con https, solo ve a qué servidor te conectas, no lo que dices. Es la diferencia entre mandar una postal (cualquiera la lee) y una carta sellada.

Probar a mano primero

Antes de programar, abre cada URL en el navegador y observa el JSON. Así sabrás qué campos esperar:

```
$ https://api.open-meteo.com/v1/forecast?latitude=21.1561&longitude=-
100.9319&current=temperature_2m,weather_code
$ https://api.frankfurter.dev/v1/latest?base=MXN&symbols=USD,EUR
$ https://date.nager.at/api/v3/PublicHolidays/2026/MX
$ https://es.wikipedia.org/api/rest_v1/page/summary/Dolores_Hidalgo
$ https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY
```

RETO PARA TI

Registra tu propia clave gratuita en api.nasa.gov (pide nombre y correo, llega al instante). Guárdala en secrets.json, NUNCA en un chat de WhatsApp ni en un documento compartido.

Ronda 2 · Cimientos seguros: el cliente HTTP

Una sola puerta de salida

Imagina un edificio con una sola entrada y un guardia. Todo el que entra o sale pasa por ahí, así que el guardia aplica las reglas de seguridad una vez y en un solo lugar. Nuestro guardia es ClienteSeguro: las 5 APIs pasan por él.

Paso 2.1 · Las excepciones

Primero los errores con nombre propio. Todos heredan de ErrorDeApp, y esto es clave para la seguridad: los errores que conocemos tienen mensajes escritos por nosotros y son seguros de mostrar; cualquier otro error (un bug, un dato raro) se mostrará como mensaje genérico para no revelar detalles internos.

lib/dominio/excepciones.dart

```
/// Todos los errores "esperados" de la app heredan de ErrorDeApp.
/// Su mensaje es seguro para mostrarse al usuario. Cualquier otro error
/// (bug, dato raro) se muestra como mensaje genérico: nunca filtramos detalles.
abstract class ErrorDeApp implements Exception {
  const ErrorDeApp();
}

class SinConexionException extends ErrorDeApp {
  const SinConexionException();
  @override
  String toString() => 'No hay conexión con el servidor. Revisa tu internet.';
}

class ErrorServidorException extends ErrorDeApp {
  final int codigo;
  const ErrorServidorException(this.codigo);
  @override
  String toString() {
    switch (codigo) {
      case 401:
      case 403:
        return 'La clave de la API no es válida o no tiene permiso ($codigo).';
      case 404:
        return 'No se encontró lo que buscabas (404).';
      case 429:
        return 'Demasiadas peticiones (429). Espera un momento e intenta de nuevo.';
      default:
        return 'El servidor respondió con error ($codigo).';
    }
  }
}

class RespuestaInvalidaException extends ErrorDeApp {
  const RespuestaInvalidaException();
  @override
  String toString() => 'El servidor respondió con datos que no entiendo.';
}

class RespuestaDemasiadoGrandeException extends ErrorDeApp {
  const RespuestaDemasiadoGrandeException();
  @override
  String toString() => 'La respuesta del servidor es demasiado grande y se descartó.';
}
```

```
class ConexionInseguraException extends ErrorDeApp {
  const ConexionInseguraException();
  @override
  String toString() => 'Por seguridad solo se permiten direcciones https://';
}

class EntradaInvalidaException extends ErrorDeApp {
  final String motivo;
  const EntradaInvalidaException(this.motivo);
  @override
  String toString() => 'Dato inválido: $motivo';
}

class ClaveFaltanteException extends ErrorDeApp {
  final String nombre;
  const ClaveFaltanteException(this.nombre);
  @override
  String toString() => 'Falta configurar la clave $nombre.';
}

/// Mensaje que SÍ puede ver el usuario.
String mensajeSeguro(Object error) => error is ErrorDeApp
    ? error.toString()
    : 'Ocurrió un error inesperado. Intenta de nuevo.';
```



```
/// Todos los errores "esperados" de la app heredan de ErrorDeApp.
/// Su mensaje es seguro para mostrarse al usuario. Cualquier otro error
/// (bug, dato raro) se muestra como mensaje genérico: nunca filtramos detalles.
abstract class ErrorDeApp implements Exception {
  const ErrorDeApp();
}

class SinConexionException extends ErrorDeApp {
  const SinConexionException();
  @override
  String toString() => 'No hay conexión con el servidor. Revisa tu internet.';
}

class ErrorServidorException extends ErrorDeApp {
  final int codigo;
  const ErrorServidorException(this.codigo);
  @override
  String toString() {
    switch (codigo) {
      case 401:
      case 403:
        return 'La clave de la API no es válida o no tiene permiso ($codigo).';
      case 404:
        return 'No se encontró lo que buscabas (404).';
      case 429:
        return 'Demasiadas peticiones (429). Espera un momento e intenta de nuevo.';
      default:
        return 'El servidor respondió con error ($codigo).';
    }
  }
}

class RespuestaInvalidaException extends ErrorDeApp {
  const RespuestaInvalidaException();
  @override
  String toString() => 'El servidor respondió con datos que no entiendo.';
}
```

> **Dato curioso / ejemplo**
> Revelar errores técnicos al usuario ayuda a los atacantes. Un mensaje como "Error en /home/app/db.dart línea 42: password=abc" le dice al atacante exactamente dónde mirar. Los desarrolladores ven los detalles en sus logs; el usuario ve un mensaje amable.

Paso 2.2 · Pruebas ROJAS del cliente

Crea primero las ayudas de prueba (respuestas simuladas de las 5 APIs y un servidor falso que las enruta por dominio). Las usaremos en todas las rondas:

test/ayudas.dart

```
// Ayudas compartidas por las pruebas (no es una prueba en sí).
import 'dart:convert';

import 'package:dolores_hoy/datos/cliente_seguro.dart';
```

```
import 'package:http/http.dart' as http;
import 'package:http/testing.dart';

http.Response jsonOk(Object cuerpo, [int estado = 200]) => http.Response.bytes(
    utf8.encode(jsonEncode(cuerpo)), estado,
    headers: {'content-type': 'application/json; charset=utf-8'});

ClienteSeguro seguro(Future<http.Response> Function(http.Request) manejador,
        {Duration? espera, int? maxBytes}) =>
    ClienteSeguro(MockClient(manejador),
        espera: espera ?? const Duration(seconds: 2),
        maxBytes: maxBytes ?? 1024 * 1024);

// Respuestas que imitan el formato documentado de cada API.
final jsonClima = {
  'current': {
    'time': '2026-10-29T14:00',
    'temperature_2m': 24.3,
    'relative_humidity_2m': 41,
    'weather_code': 1,
    'wind_speed_10m': 9.4,
  }
};

final jsonCambio = {
  'amount': 1.0,
  'base': 'MXN',
  'date': '2026-10-28',
  'rates': {'EUR': 0.05, 'USD': 0.0555},
};

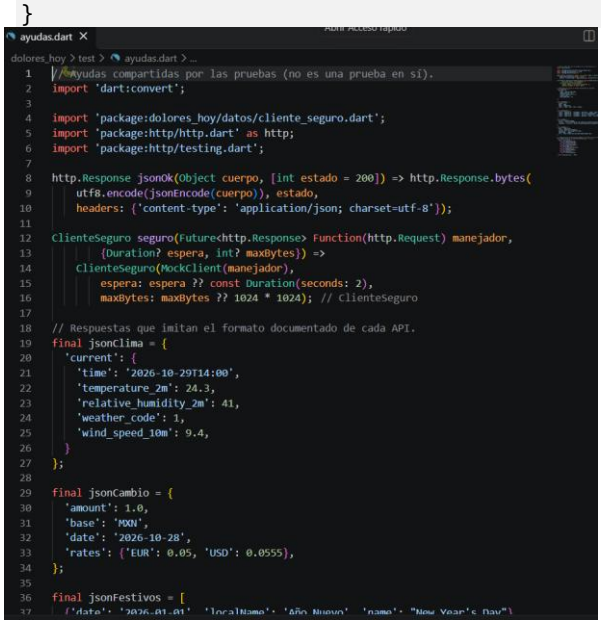
final jsonFestivos = [
  {'date': '2026-01-01', 'localName': 'Año Nuevo', 'name': "New Year's Day"},
  {'date': '2026-09-16', 'localName': 'Día de la Independencia', 'name': 'Independence Day'},
  {'date': '2026-11-16', 'localName': 'Día de la Revolución', 'name': 'Revolution Day'},
  {'date': '2026-12-25', 'localName': 'Navidad', 'name': 'Christmas Day'},
];

final jsonWiki = {
  'title': 'Dolores Hidalgo',
  'extract': 'Dolores Hidalgo es una ciudad del estado de Guanajuato, México.',
  'thumbnail': {'source': 'https://upload.wikimedia.org/ejemplo.jpg', 'width': 320},
};

final jsonNasa = {
  'date': '2026-10-29',
  'title': 'Nebulosa de ejemplo',
  'explanation': 'Una imagen de prueba.',
  'media_type': 'image',
  'url': 'https://apod.nasa.gov/apod/image/ejemplo.jpg',
};

/// Servidor falso que enruta por dominio: las 5 APIs en una sola función.
Future<http.Response> servidorFalso(http.Request r) async {
  switch (r.url.host) {
    case 'api.open-meteo.com':
      return jsonOk(jsonClima);
    case 'api.frankfurter.dev':
      return jsonOk(jsonCambio);
    case 'date.nager.at':
      return jsonOk(jsonFestivos);
    case 'es.wikipedia.org':
      return jsonOk(jsonWiki);
```

```
    case 'api.nasa.gov':
      return jsonOk(jsonNasa);
  }
  return http.Response('', 404);
}
```



> ⚠ **Cuidado**
> jsonOk crea la respuesta con bytes UTF-8 y la cabecera charset=utf-8. Sin eso, los acentos ("Año Nuevo") se corrompen en las pruebas. Es un detalle de codificación de caracteres que da muchos dolores de cabeza.

Ahora las pruebas del cliente. Léelas con calma: cada una describe una defensa.

test/cliente_seguro_test.dart

```
import 'dart:io';

import 'package:dolores_hoy/datos/cliente_seguro.dart';
import 'package:dolores_hoy/dominio/excepciones.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:http/http.dart' as http;

import 'ayudas.dart';

void main() {
  final https = Uri.parse('https://ejemplo.com/datos');

  test('rechaza direcciones http:// sin siquiera hacer la petición', () {
    var llamado = false;
    final c = seguro((_) async {
      llamado = true;
      return jsonOk({});
    });
    expect(c.obtenerJson(Uri.parse('http://ejemplo.com')),
        throwsA(isA<ConexionInseguraException>()));
    expect(llamado, false);
  });

  test('devuelve el JSON decodificado (con acentos)', () async {
    final c = seguro((_) async => jsonOk({'nombre': 'Año Nuevo'}));
    final r = await c.obtenerJson(https) as Map<String, dynamic>;
```

```
    expect(r['nombre'], 'Año Nuevo');
  });

  test('envía cabeceras Accept y User-Agent', () async {
    final c = seguro((req) async {
      expect(req.headers['Accept'], 'application/json');
      expect(req.headers['User-Agent'], contains('DoloresHoy'));
      return jsonOk({});
    });
    await c.obtenerJson(https);
  });

  test('429 produce un mensaje de límite de peticiones', () {
    final c = seguro((_) async => jsonOk({}, 429));
    expect(
        c.obtenerJson(https),
        throwsA(isA<ErrorServidorException>().having((e) => e.toString(), 'mensaje',
contains('Demasiadas'))));
  });

  test('500 lanza ErrorServidorException con el código', () {
    final c = seguro((_) async => jsonOk({}, 500));
    expect(c.obtenerJson(https),
        throwsA(isA<ErrorServidorException>().having((e) => e.codigo, 'codigo', 500)));
  });

  test('sin red lanza SinConexionException', () {
    final c = seguro((_) async => throw const SocketException('x'));
    expect(c.obtenerJson(https), throwsA(isA<SinConexionException>()));
  });

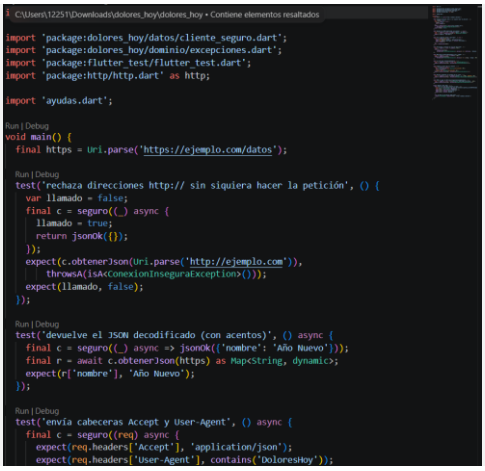
  test('timeout se trata como sin conexión', () {
    final c = seguro((_) async {
      await Future.delayed(const Duration(milliseconds: 200));
      return jsonOk({});
    }, espera: const Duration(milliseconds: 20));
    expect(c.obtenerJson(https), throwsA(isA<SinConexionException>()));
  });

  test('descarta respuestas más grandes que el límite', () {
    final c = seguro((_) async => jsonOk({'relleno': 'x' * 500}), maxBytes: 100);
    expect(c.obtenerJson(https), throwsA(isA<RespuestaDemasiadoGrandeException>()));
  });

  test('HTML en lugar de JSON lanza RespuestaInvalidaException', () {
    final c = seguro((_) async => http.Response('<html>login wifi</html>', 200));
    expect(c.obtenerJson(https), throwsA(isA<RespuestaInvalidaException>()));
  });

  group('redactar (para logs)', () {
    test('oculta la clave pero conserva el resto', () {
      final u = Uri.parse('https://api.nasa.gov/planetary/apod?api_key=SECRETO123&dia=hoy');
      final texto = ClienteSeguro.redactar(u);
      expect(texto, isNot(contains('SECRETO123')));
      expect(texto, contains('api_key=***'));
      expect(texto, contains('dia=hoy'));
    });

    test('una URL sin parámetros queda igual', () {
      expect(ClienteSeguro.redactar(https), 'https://ejemplo.com/datos');
    });
  });
}
```



ROJO: escribe la prueba (debe FALLAR)
Ejecuta flutter test. Falla porque cliente_seguro.dart no existe. Perfecto.

Paso 2.3 · VERDE: implementar el guardia

lib/datos/cliente_seguro.dart

```
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as http;

import '../dominio/excepciones.dart';

/// Único punto de salida a internet de toda la app.
/// Aquí viven las defensas: https obligatorio, timeout, límite de tamaño,
/// JSON validado y errores traducidos. Las 5 APIs pasan por aquí.
class ClienteSeguro {
  final http.Client _cliente;
  final Duration espera;
  final int maxBytes;

  ClienteSeguro(this._cliente,
      {this.espera = const Duration(seconds: 8), this.maxBytes = 1024 * 1024});

  static const _cabeceras = {
    'Accept': 'application/json',
    'User-Agent': 'DoloresHoy/1.0 (curso Flutter UTNG)',
  };

  Future<dynamic> obtenerJson(Uri uri) async {
    if (uri.scheme != 'https') throw const ConexionInseguraException();

    http.Response r;
    try {
      r = await _cliente.get(uri, headers: _cabeceras).timeout(espera);
    } on SocketException {
      throw const SinConexionException();
    } on TimeoutException {
      throw const SinConexionException();
    } on http.ClientException {
      throw const SinConexionException();
    }
```

```
    if (r.statusCode < 200 || r.statusCode >= 300) {
      throw ErrorServidorException(r.statusCode);
    }
    if (r.bodyBytes.length > maxBytes) {
      throw const RespuestaDemasiadoGrandeException();
    }
    try {
      return jsonDecode(utf8.decode(r.bodyBytes));
    } catch (_) {
      throw const RespuestaInvalidaException();
    }
  }

  /// Versión de la URL segura para escribir en logs: oculta claves.
  static String redactar(Uri uri) {
    const secretos = {'api_key', 'apikey', 'key', 'token', 'access_token'};
    final params = uri.queryParameters.map(
      (k, v) => MapEntry(k, secretos.contains(k.toLowerCase()) ? '***' : v),
    );
    return uri.replace(queryParameters: params.isEmpty ? null : params).toString();
  }
}

/// Envuelve la conversión JSON→objeto: si el servidor cambia su formato,
/// el usuario ve un error claro y no un crash.
T convertir<T>(T Function() f) {
  try {
    return f();
  } catch (_) {
    throw const RespuestaInvalidaException();
  }
}
```



```
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as http;

import '../dominio/excepciones.dart';

/// Único punto de salida a internet de toda la app.
/// Aquí viven las defensas: https obligatorio, timeout, límite de tamaño,
/// JSON validado y errores traducidos. Las 5 APIs pasan por aquí.
class ClienteSeguro {
  final http.Client _cliente;
  final Duration espera;
  final int maxBytes;

  ClienteSeguro(this._cliente,
      {this.espera = const Duration(seconds: 8), this.maxBytes = 1024 * 1024});

  static const _cabeceras = {
    'Accept': 'application/json',
    'User-Agent': 'DoloresHoy/1.0 (curso Flutter UTNG)',
  };

  Future<dynamic> obtenerJson(Uri uri) async {
    if (uri.scheme != 'https') throw const ConexionInseguraException();

    http.Response r;
    try {
      r = await _cliente.get(uri, headers: _cabeceras).timeout(espera);
    } on SocketException {
      throw const SinConexionException();
    } on TimeoutException {
      throw const SinConexionException();
    } on http.ClientException {
      throw const SinConexionException();
```

```
$ flutter test test/cliente_seguro_test.dart
```

Las 6 defensas, explicadas una por una

3. HTTPS obligatorio: si alguien (o un error tuyo) escribe http://, se rechaza ANTES de enviar nada. Ningún dato sale por un canal sin cifrar.
4. Timeout de 8 segundos: sin él, un servidor lento deja tu app esperando para siempre, y un atacante podría usar eso para bloquearla.

5. Límite de tamaño (1 MB): si un servidor comprometido te manda 500 MB de basura, la app se quedaría sin memoria. Descartamos respuestas absurdas.
6. JSON validado: si llega HTML (como la página de login de un Wi-Fi público), no se trata como datos.
7. Errores traducidos: el 429 (demasiadas peticiones) y el 403 (clave inválida) tienen mensajes claros y útiles.
8. Cabeceras honestas: User-Agent que dice quién eres. Wikipedia lo pide, y es cortesía con los servicios gratuitos.

Dato curioso / ejemplo

El método redactar() ya está listo para los logs: convierte api_key=SECRETO en api_key=***. Úsalo SIEMPRE que imprimas una URL, aunque sea solo para depurar. Los logs terminan en reportes de errores, capturas de pantalla y chats de soporte.

REFACTOR: ordena sin romper nada

Fíjate en convertir<T>(): una función pequeña que cada servicio usa para convertir JSON en objetos. Si el servidor cambia su formato, el resultado es RespuestaInvalidaException con mensaje claro, no un crash.

INSIGNIA DESBLOQUEADA: Guardia de la puerta

Ronda 3 · API 1: Clima (Open-Meteo)

La más sencilla: sin clave y con una respuesta limpia. Es ideal para aprender el patrón que repetiremos cuatro veces más: modelo → servicio → pruebas.

La respuesta que esperamos

```
{
  "current": {
    "time": "2026-10-29T14:00",
    "temperature_2m": 24.3,
    "relative_humidity_2m": 41,
    "weather_code": 1,
    "wind_speed_10m": 9.4
  }
}
```

Ciclo 3.1 · ROJO: las pruebas

Crea test/servicios_test.dart. Empezamos con el grupo del clima (el archivo completo se va construyendo ronda por ronda). Verifica el dominio, la latitud enviada, la validación de entrada y qué pasa si el JSON viene incompleto:

test/servicios_test.dart (arranque + grupo 1)

```
import 'package:dolores_hoy/datos/config_claves.dart';
import 'package:dolores_hoy/datos/servicios.dart';
import 'package:dolores_hoy/dominio/excepciones.dart';
import 'package:dolores_hoy/dominio/modelos.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

void main() {
  group('API 1 · Clima (Open-Meteo)', () {
    test('convierte la respuesta en Clima', () async {
      final s = ServicioClima(seguro((req) async {
        expect(req.url.host, 'api.open-meteo.com');
        expect(req.url.queryParameters['latitude'], '21.1561');
        return jsonOk(jsonClima);
      }));
      final c = await s.actual();
      expect(c.temperatura, 24.3);
      expect(c.humedad, 41);
      expect(c.codigo, 1);
    });

    test('rechaza coordenadas fuera de rango', () {
      final s = ServicioClima(seguro((_) async => jsonOk(jsonClima)));
      expect(s.actual(lat: 200), throwsA(isA<EntradaInvalidaException>()));
    });

    test('si falta el bloque current lanza RespuestaInvalida', () {
      final s = ServicioClima(seguro((_) async => jsonOk({'otra': 1})));
      expect(s.actual(), throwsA(isA<RespuestaInvalidaException>()));
    });

    test('describirClima traduce códigos', () {
```

```
      expect(describirClima(0).$2, 'Despejado');
      expect(describirClima(63).$2, 'Lluvia');
      expect(describirClima(95).$2, 'Tormenta');
    });
  });

}
```



```dart
import 'package:dolores_hoy/datos/config_claves.dart';
import 'package:dolores_hoy/datos/servicios.dart';
import 'package:dolores_hoy/dominio/excepciones.dart';
import 'package:dolores_hoy/dominio/modelos.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

Run | Debug
void main() {
  Run | Debug
  group('API 1 · Clima (Open-Meteo)', () {
    Run | Debug
    test('convierte la respuesta en Clima', () async {
      final s = ServicioClima(seguro((req) async {
        expect(req.url.host, 'api.open-meteo.com');
        expect(req.url.queryParameters['latitude'], '21.1561');
        return jsonOk(jsonClima);
      })); // ServicioClima
      final c = await s.actual();
      expect(c.temperatura, 24.3);
      expect(c.humedad, 41);
      expect(c.codigo, 1);
    }); // test('convierte la respuesta en Clima')

    Run | Debug
    test('rechaza coordenadas fuera de rango', () {
      final s = ServicioClima(seguro((_) async => jsonOk(jsonClima)));
      expect(s.actual(lat: 200), throwsA(isA<EntradaInvalidaException>()));
    }); // test('rechaza coordenadas fuera de rango')

    Run | Debug
    test('si falta el bloque current lanza RespuestaInvalida', () {
      final s = ServicioClima(seguro((_) async => jsonOk({'otra': 1})));
      expect(s.actual(), throwsA(isA<RespuestaInvalidaException>()));
    }); // test('si falta el bloque current lanza RespuestaInvalida')
```

Ciclo 3.2 · VERDE: el modelo

lib/dominio/modelos.dart (parte 1)

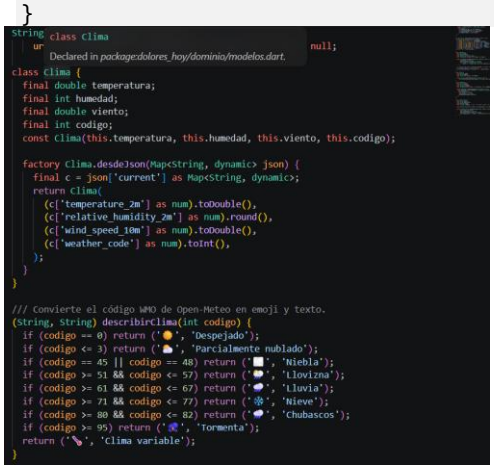
```dart
/// Solo aceptamos URLs https (por ejemplo para imágenes que vienen de una API).
String? soloHttps(Object? url) =>
    url is String && url.startsWith('https://') ? url : null;

class Clima {
  final double temperatura;
  final int humedad;
  final double viento;
  final int codigo;
  const Clima(this.temperatura, this.humedad, this.viento, this.codigo);

  factory Clima.desdeJson(Map<String, dynamic> json) {
    final c = json['current'] as Map<String, dynamic>;
    return Clima(
      (c['temperature_2m'] as num).toDouble(),
      (c['relative_humidity_2m'] as num).round(),
      (c['wind_speed_10m'] as num).toDouble(),
      (c['weather_code'] as num).toInt(),
    );
  }
}

/// Convierte el código WMO de Open-Meteo en emoji y texto.
(String, String) describirClima(int codigo) {
  if (codigo == 0) return ('☀️', 'Despejado');
  if (codigo <= 3) return ('⛅', 'Parcialmente nublado');
  if (codigo == 45 || codigo == 48) return ('🌫️', 'Niebla');
  if (codigo >= 51 && codigo <= 57) return ('🌦️', 'Llovizna');
```

```
  if (codigo >= 61 && codigo <= 67) return ('🌧', 'Lluvia');
  if (codigo >= 71 && codigo <= 77) return ('❄', 'Nieve');
  if (codigo >= 80 && codigo <= 82) return ('🌧', 'Chubascos');
  if (codigo >= 95) return ('⛈', 'Tormenta');
  return ('🌡', 'Clima variable');
}
```



> **Dato curioso / ejemplo**
> describirClima devuelve un "record" de Dart 3: (String, String). Se lee con .$1 y .$2 o se desestructura: final (emoji, texto) = describirClima(63). Es como devolver dos cosas por la ventanilla en vez de una.

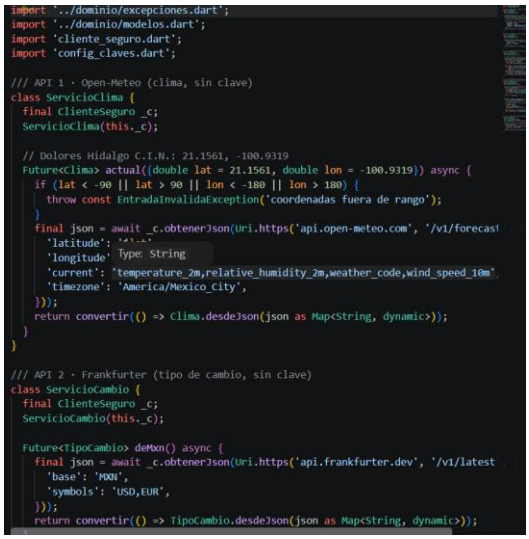
Ciclo 3.3 · VERDE: el servicio

lib/datos/servicios.dart (encabezado + API 1)

```
import '../dominio/excepciones.dart';
import '../dominio/modelos.dart';
import 'cliente_seguro.dart';
import 'config_claves.dart';

/// API 1 · Open-Meteo (clima, sin clave)
class ServicioClima {
  final ClienteSeguro _c;
  ServicioClima(this._c);

  // Dolores Hidalgo C.I.N.: 21.1561, -100.9319
  Future<Clima> actual({double lat = 21.1561, double lon = -100.9319}) async {
    if (lat < -90 || lat > 90 || lon < -180 || lon > 180) {
      throw const EntradaInvalidaException('coordenadas fuera de rango');
    }
    final json = await _c.obtenerJson(Uri.https('api.open-meteo.com', '/v1/forecast', {
      'latitude': '$lat',
      'longitude': '$lon',
      'current': 'temperature_2m,relative_humidity_2m,weather_code,wind_speed_10m',
      'timezone': 'America/Mexico_City',
    }));
    return convertir(() => Clima.desdeJson(json as Map<String, dynamic>));
  }
}
```



```dart
import '../dominio/excepciones.dart';
import '../dominio/modelos.dart';
import 'cliente_seguro.dart';
import 'config_claves.dart';

/// API 1 · Open-Meteo (clima, sin clave)
class ServicioClima {
  final ClienteSeguro _c;
  ServicioClima(this._c);

  // Dolores Hidalgo C.I.N.: 21.1561, -100.9319
  Future<Clima> actual({double lat = 21.1561, double lon = -100.9319}) async {
    if (lat < -90 || lat > 90 || lon < -180 || lon > 180) {
      throw const EntradaInvalidaException('coordenadas fuera de rango');
    }
    final json = await _c.obtenerJson(Uri.https('api.open-meteo.com', '/v1/forecast
      'latitude': '
      'longitude'  Type: String
      'current': 'temperature_2m,relative_humidity_2m,weather_code,wind_speed_10m'
      'timezone': 'America/Mexico_City',
    }));
    return convertir(() => Clima.desdeJson(json as Map<String, dynamic>));
  }
}

/// API 2 · Frankfurter (tipo de cambio, sin clave)
class ServicioCambio {
  final ClienteSeguro _c;
  ServicioCambio(this._c);

  Future<TipoCambio> deMxn() async {
    final json = await _c.obtenerJson(Uri.https('api.frankfurter.dev', '/v1/latest
      'base': 'MXN',
      'symbols': 'USD,EUR',
    }));
    return convertir(() => TipoCambio.desdeJson(json as Map<String, dynamic>));
```

> ⚠ **Cuidado**
>
> Uri.https(host, ruta, parametros) construye la dirección SIEMPRE con https y codifica los parámetros por ti. Mucho mejor que armar el texto a mano con "+" o interpolación: ahí es donde nacen los errores y las inyecciones.

> **REFACTOR: ordena sin romper nada**
>
> Validamos la latitud y longitud ANTES de pedir. Las coordenadas son datos de entrada: hoy las pone el programador, mañana podrían venir del GPS o de un campo de texto. La validación ya está lista.

INSIGNIA DESBLOQUEADA: Meteorólogo de Dolores

Ronda 4 · API 2: Tipo de cambio (Frankfurter)

El truco matemático

La API responde cuántas unidades de otra moneda se obtienen por 1 peso: "1 MXN = 0.05 USD". Pero nosotros queremos saber cuántos pesos cuesta 1 dólar. Es la operación inversa: 1 / 0.05 = 20 pesos. Como ir a una casa de cambio: te dicen "te doy 20 pesos por dólar", no "te doy 0.05 dólares por peso".

```
{
  "amount": 1.0,
  "base": "MXN",
  "date": "2026-10-28",
  "rates": { "EUR": 0.05, "USD": 0.0555 }
}
```

Ciclo 4.1 · ROJO

test/servicios_test.dart (grupo 2)

```
  group('API 2 · Tipo de cambio (Frankfurter)', () {
    test('calcula pesos por dólar invirtiendo la tasa', () async {
      final s = ServicioCambio(seguro((_) async => jsonOk({
            'base': 'MXN', 'date': '2026-10-28', 'rates': {'USD': 0.05, 'EUR': 0.04}
          })));
      final t = await s.deMxn();
      expect(t.pesosPorUnidad('USD'), closeTo(20.0, 0.0001));
      expect(t.pesosPorUnidad('EUR'), closeTo(25.0, 0.0001));
    });
  });
```

```dart
/// API 2 · Frankfurter (tipo de cambio, sin clave)
class ServicioCambio {
  final ClienteSeguro _c;
  ServicioCambio(this._c);

  Future<TipoCambio> deMxn() async {
    final json = await _c.obtenerJson(Uri.https('api.frankfurter.dev', '/v1/latest', {
      'base': 'MXN',
      'symbols': 'USD,EUR',
    }));
    return convertir(() => TipoCambio.desdeJson(json as Map<String, dynamic>));
  }
}
```

Ciclo 4.2 · VERDE

lib/dominio/modelos.dart (agrega)

```
class TipoCambio {
  final String base;
  final DateTime fecha;
  final Map<String, double> tasas;
  const TipoCambio(this.base, this.fecha, this.tasas);

  factory TipoCambio.desdeJson(Map<String, dynamic> json) => TipoCambio(
        json['base'] as String,
```

```
        DateTime.parse(json['date'] as String),
        (json['rates'] as Map<String, dynamic>)
            .map((k, v) => MapEntry(k, (v as num).toDouble())),
      );

  /// Cuántos pesos cuesta 1 unidad de [moneda]. Si 1 MXN = 0.05 USD,
  /// entonces 1 USD = 1 / 0.05 = 20 MXN.
  double pesosPorUnidad(String moneda) => 1 / tasas[moneda]!;
}
```

```dart
class TipoCambio {
  final String base;
  final DateTime fecha;
  final Map<String, double> tasas;
  const TipoCambio(this.base, this.fecha, this.tasas);

  factory TipoCambio.desdeJson(Map<String, dynamic> json) => TipoCambio(
        json['base'] as String,
        DateTime.parse(json['date'] as String),
        (json['rates'] as Map<String, dynamic>)
            .map((k, v) => MapEntry(k, (v as num).toDouble())),
      ); // TipoCambio

  /// Cuántos pesos cuesta 1 unidad de [moneda]. Si 1 MXN = 0.05 USD,
  /// entonces 1 USD = 1 / 0.05 = 20 MXN.
  double pesosPorUnidad(String moneda) => 1 / tasas[moneda]!;
}
```

lib/datos/servicios.dart (agrega)

```
/// API 2 · Frankfurter (tipo de cambio, sin clave)
class ServicioCambio {
  final ClienteSeguro _c;
  ServicioCambio(this._c);

  Future<TipoCambio> deMxn() async {
    final json = await _c.obtenerJson(Uri.https('api.frankfurter.dev', '/v1/latest', {
      'base': 'MXN',
      'symbols': 'USD,EUR',
    }));
    return convertir(() => TipoCambio.desdeJson(json as Map<String, dynamic>));
  }
}
```

```dart
/// API 2 · Frankfurter (tipo de cambio, sin clave)
class ServicioCambio {
  final ClienteSeguro _c;
  ServicioCambio(this._c);

  Future<TipoCambio> deMxn() async {
    final json = await _c.obtenerJson(Uri.https('api.frankfurter.dev', '/v1/latest', {
      'base': 'MXN',
      'symbols': 'USD,EUR',
    }));
    return convertir(() => TipoCambio.desdeJson(json as Map<String, dynamic>));
  }
}
```

> **⚠ Cuidado**
> Las tasas de cambio son decimales y aquí usamos double solo para MOSTRAR (dos decimales). Si fueras a hacer una compra real, usarías centavos enteros como en el punto de venta. Regla: para mostrar, double está bien; para cobrar, enteros.

> **Dato curioso / ejemplo**
> Frankfurter publica las tasas del Banco Central Europeo, que se actualizan en días hábiles. Por eso el sábado verás la fecha del viernes: no es un error, es el último dato oficial disponible.

INSIGNIA DESBLOQUEADA: Casa de cambio

Ronda 5 · API 3: Días festivos (Nager.Date)

El problema de probar con fechas

Queremos "los próximos festivos desde hoy". Pero si la prueba usa la fecha real, un test que pasa en octubre falla en diciembre. Un test que depende del calendario es un test con fecha de caducidad. La solución: inyectar la fecha de hoy como parámetro opcional. En la app no se pasa (usa la real); en las pruebas se fija.

```
[
  { "date": "2026-01-01", "localName": "Año Nuevo", "name": "New Year's Day", ... },
  { "date": "2026-09-16", "localName": "Día de la Independencia", ... },
  ...
]
```

Ciclo 5.1 · ROJO

test/servicios_test.dart (grupo 3)

```
  group('API 3 · Festivos (Nager.Date)', () {
    test('devuelve solo los próximos, en orden', () async {
      final s = ServicioFestivos(seguro((req) async {
        expect(req.url.path, '/api/v3/PublicHolidays/2026/MX');
        return jsonOk(jsonFestivos);
      }));
      final r = await s.proximos(hoy: DateTime(2026, 10, 29), cuantos: 2);
      expect(r.map((f) => f.nombre), ['Día de la Revolución', 'Navidad']);
    });

    test('incluye el festivo de hoy mismo', () async {
      final s = ServicioFestivos(seguro((_) async => jsonOk(jsonFestivos)));
      final r = await s.proximos(hoy: DateTime(2026, 12, 25), cuantos: 5);
      expect(r.single.nombre, 'Navidad');
    });

    test('rechaza años absurdos', () {
      final s = ServicioFestivos(seguro((_) async => jsonOk(jsonFestivos)));
      expect(s.proximos(anio: 3000), throwsA(isA<EntradaInvalidaException>()));
    });
  });
```

```dart
Run | Debug
group('API 3 · Festivos (Nager.Date)', () {
  Run | Debug
  test('devuelve solo los próximos, en orden', () async {
    final s = ServicioFestivos(seguro((req) async {
      expect(req.url.path, '/api/v3/PublicHolidays/2026/MX');
      return jsonOk(jsonFestivos);
    })); // ServicioFestivos
    final r = await s.proximos(hoy: DateTime(2026, 10, 29), cuantos: 2);
    expect(r.map((f) => f.nombre), ['Día de la Revolución', 'Navidad']);
  }); // test('devuelve solo los próximos, en orden')

  Run | Debug
  test('incluye el festivo de hoy mismo', () async {
    final s = ServicioFestivos(seguro((_) async => jsonOk(jsonFestivos)));
    final r = await s.proximos(hoy: DateTime(2026, 12, 25), cuantos: 5);
    expect(r.single.nombre, 'Navidad');
  }); // test('incluye el festivo de hoy mismo')

  Run | Debug
  test('rechaza años absurdos', () {
    final s = ServicioFestivos(seguro((_) async => jsonOk(jsonFestivos)));
    expect(s.proximos(anio: 3000), throwsA(isA<EntradaInvalidaException>()));
  }); // test('rechaza años absurdos')
}); // group('API 3 · Festivos (Nager.Date)')
```

Ciclo 5.2 · VERDE

lib/dominio/modelos.dart (agrega)

```dart
class Festivo {
  final DateTime fecha;
  final String nombre;
  const Festivo(this.fecha, this.nombre);

  factory Festivo.desdeJson(Map<String, dynamic> json) =>
      Festivo(DateTime.parse(json['date'] as String), json['localName'] as String);
}
```


```dart
class Festivo {
  final DateTime fecha;
  final String nombre;
  const Festivo(this.fecha, this.nombre);

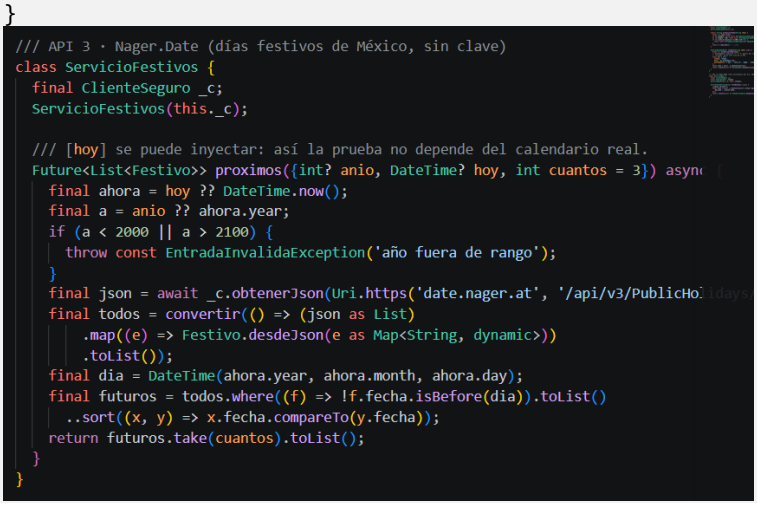
  factory Festivo.desdeJson(Map<String, dynamic> json) =>
      Festivo(DateTime.parse(json['date'] as String), json['localName'] as String);
}
```


lib/datos/servicios.dart (agrega)

```dart
/// API 3 · Nager.Date (días festivos de México, sin clave)
class ServicioFestivos {
  final ClienteSeguro _c;
  ServicioFestivos(this._c);

  /// [hoy] se puede inyectar: así la prueba no depende del calendario real.
  Future<List<Festivo>> proximos({int? anio, DateTime? hoy, int cuantos = 3}) async {
    final ahora = hoy ?? DateTime.now();
    final a = anio ?? ahora.year;
    if (a < 2000 || a > 2100) {
      throw const EntradaInvalidaException('año fuera de rango');
    }
    final json = await _c.obtenerJson(Uri.https('date.nager.at',
'/api/v3/PublicHolidays/$a/MX'));
    final todos = convertir(() => (json as List)
        .map((e) => Festivo.desdeJson(e as Map<String, dynamic>))
```

```
        .toList());
    final dia = DateTime(ahora.year, ahora.month, ahora.day);
    final futuros = todos.where((f) => !f.fecha.isBefore(dia)).toList()
      ..sort((x, y) => x.fecha.compareTo(y.fecha));
    return futuros.take(cuantos).toList();
  }
}
```



```
/// API 3 · Nager.Date (días festivos de México, sin clave)
class ServicioFestivos {
  final ClienteSeguro _c;
  ServicioFestivos(this._c);

  /// [hoy] se puede inyectar: así la prueba no depende del calendario real.
  Future<List<Festivo>> proximos({int? anio, DateTime? hoy, int cuantos = 3}) async {
    final ahora = hoy ?? DateTime.now();
    final a = anio ?? ahora.year;
    if (a < 2000 || a > 2100) {
      throw const EntradaInvalidaException('año fuera de rango');
    }
    final json = await _c.obtenerJson(Uri.https('date.nager.at', '/api/v3/PublicHolidays/$a/MX'));
    final todos = convertir(() => (json as List)
        .map((e) => Festivo.desdeJson(e as Map<String, dynamic>))
        .toList());
    final dia = DateTime(ahora.year, ahora.month, ahora.day);
    final futuros = todos.where((f) => !f.fecha.isBefore(dia)).toList()
      ..sort((x, y) => x.fecha.compareTo(y.fecha));
    return futuros.take(cuantos).toList();
  }
}
```

> ⚠ **Cuidado**
> Observa !f.fecha.isBefore(dia): el festivo de HOY debe aparecer. La segunda prueba existe justo para atrapar ese error clásico de "menor o igual" (se llama error de frontera, o "off by one").

> 💡 **Dato curioso / ejemplo**
> El año se valida entre 2000 y 2100. ¿Por qué? Porque ese número va DENTRO de la dirección. Aunque hoy lo ponga el programador, un valor sin validar es una puerta abierta mañana.

🏅 INSIGNIA DESBLOQUEADA: Guardián del calendario

Ronda 6 · API 4: Wikipedia y la entrada del usuario

El peligro de los textos del usuario

Hasta ahora todos los datos los controlaba el programador. Aquí el usuario escribe un tema y ese texto viajará DENTRO de la dirección. Todo lo que escribe un usuario es potencialmente peligroso, no porque sea malo, sino porque puede ser raro, enorme o malicioso.

Ataque de ejemplo: inyección de ruta (path injection)

Si construyes la dirección así: "https://es.wikipedia.org/api/rest_v1/page/summary/" + tema, y el usuario escribe "../../admin", ¡la dirección apunta a otro lugar de la API! Es como si en la tienda dijeras "sube al piso 3" y alguien responde "mejor llévame a la bodega". Nuestro código debe garantizar que el texto del usuario SIEMPRE sea solo el nombre del tema.

La defensa en tres capas

9. Validar: no vacío, máximo 80 caracteres, sin caracteres de control (como saltos de línea).
10. Codificar: construir la ruta con pathSegments, que convierte "/" y "?" en %2F y %3F dentro del segmento. El texto nunca puede crear segmentos nuevos.
11. Verificar la salida: la miniatura que devuelve la API solo se acepta si es https.

Ciclo 6.1 · ROJO

test/servicios_test.dart (grupo 4)

```
group('API 4 · Wikipedia', () {
  test('pide el resumen del tema con guiones bajos', () async {
    final s = ServicioWikipedia(seguro((req) async {
      expect(req.url.host, 'es.wikipedia.org');
      expect(req.url.pathSegments.last, 'Dolores_Hidalgo');
      return jsonOk(jsonWiki);
    }));
    final r = await s.resumen('Dolores Hidalgo');
    expect(r.titulo, 'Dolores Hidalgo');
    expect(r.urlImagen, startsWith('https://'));
  });

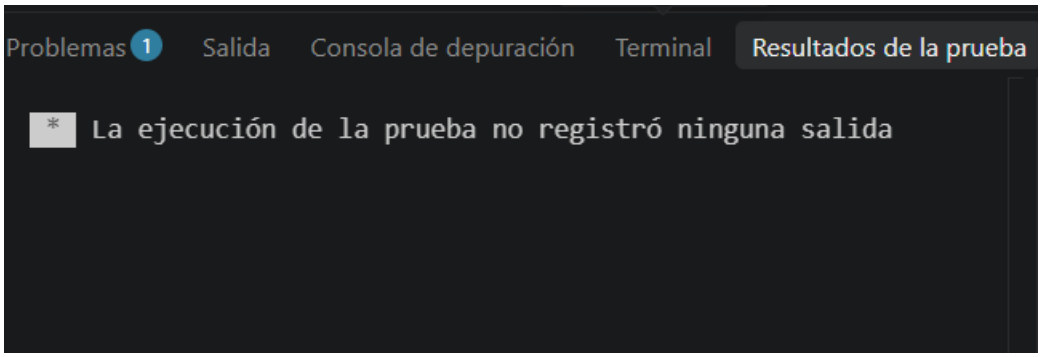
  test('un tema con "/" NO puede escapar de la ruta (path injection)', () async {
    final s = ServicioWikipedia(seguro((req) async {
      // Siempre exactamente 5 segmentos: api/rest_v1/page/summary/<tema>
      expect(req.url.pathSegments.length, 5);
      expect(req.url.pathSegments.last, '../../admin');
      return jsonOk(jsonWiki);
    }));
    await s.resumen('../../admin');
  });

  test('rechaza vacío, demasiado largo y caracteres de control', () {
    expect(() => ServicioWikipedia.normalizarTema('   '),
        throwsA(isA<EntradaInvalidaException>()));
    expect(() => ServicioWikipedia.normalizarTema('a' * 81),
        throwsA(isA<EntradaInvalidaException>()));
    expect(() => ServicioWikipedia.normalizarTema('hola\nmundo'),
        throwsA(isA<EntradaInvalidaException>()));
```

```
    });

    test('ignora miniaturas que no sean https', () async {
      final s = ServicioWikipedia(seguro((_) async => jsonOk({
            'title': 'X', 'extract': 'Y', 'thumbnail': {'source': 'http://inseguro.com/a.jpg'}
          })));
      expect((await s.resumen('X')).urlImagen, isNull);
    });
  });
```





La segunda prueba es el corazón del capítulo: aunque el usuario escriba "../../admin", la ruta tiene exactamente 5 segmentos y el último es el texto completo, tal cual, codificado.

Ciclo 6.2 · VERDE

lib/dominio/modelos.dart (agrega)

```
class ResumenWiki {
  final String titulo;
  final String extracto;
  final String? urlImagen;
  const ResumenWiki(this.titulo, this.extracto, this.urlImagen);

  factory ResumenWiki.desdeJson(Map<String, dynamic> json) {
    final miniatura = json['thumbnail'] as Map<String, dynamic>?;
    return ResumenWiki(
      json['title'] as String,
      json['extract'] as String,
      soloHttps(miniatura?['source']),
    );
  }
}
```

```dart
class ResumenWiki {
  final String titulo;
  final String extracto;
  final String? urlImagen;
  const ResumenWiki(this.titulo, this.extracto, this.urlImagen);

  factory ResumenWiki.desdeJson(Map<String, dynamic> json) {
    final miniatura = json['thumbnail'] as Map<String, dynamic>?;
    return ResumenWiki(
      json['title'] as String,
      json['extract'] as String,
      soloHttps(miniatura?['source']),
    );
  }
}
```

lib/datos/servicios.dart (agrega)

```dart
/// API 4 · Wikipedia en español (resumen de un tema, sin clave).
/// Recibe TEXTO del usuario: por eso se valida y se codifica.
class ServicioWikipedia {
  final ClienteSeguro _c;
  ServicioWikipedia(this._c);

  static String normalizarTema(String tema) {
    final t = tema.trim();
    if (t.isEmpty) throw const EntradaInvalidaException('escribe un tema');
    if (t.length > 80) throw const EntradaInvalidaException('tema demasiado largo');
    if (t.contains(RegExp(r'[\x00-\x1F]'))) {
      throw const EntradaInvalidaException('caracteres no permitidos');
    }
    return t.replaceAll(' ', '_');
  }

  Future<ResumenWiki> resumen(String tema) async {
    final t = normalizarTema(tema);
    // pathSegments codifica "/" y "?" dentro del segmento: no se puede
    // "escapar" hacia otra ruta de la API.
    final uri = Uri(
      scheme: 'https',
      host: 'es.wikipedia.org',
      pathSegments: ['api', 'rest_v1', 'page', 'summary', t],
    );
    final json = await _c.obtenerJson(uri);
    return convertir(() => ResumenWiki.desdeJson(json as Map<String, dynamic>));
  }
```

```dart
class ServicioWikipedia {
  final ClienteSeguro _c;
  ServicioWikipedia(this._c);

  static String normalizarTema(String tema) {
    final t = tema.trim();
    if (t.isEmpty) throw const EntradaInvalidaException('escribe un tema');
    if (t.length > 80) throw const EntradaInvalidaException('tema demasiado largo');
    if (t.contains(RegExp(r'[\x00-\x1F]'))) {
      throw const EntradaInvalidaException('caracteres no permitidos');
    }
    return t.replaceAll(' ', '_');
  }

  Future<ResumenWiki> resumen(String tema) async {
    final t = normalizarTema(tema);
    // pathSegments codifica "/" y "?" dentro del segmento: no se puede
    // "escapar" hacia otra ruta de la API.
    final uri = Uri(
      scheme: 'https',
      host: 'es.wikipedia.org',
      pathSegments: ['api', 'rest_v1', 'page', 'summary', t],
    );
    final json = await _c.obtenerJson(uri);
    return convertir(() => ResumenWiki.desdeJson(json as Map<String, dynamic>));
  }
}
```

Dato curioso / ejemplo

Este tipo de ataques existe desde hace décadas y sigue apareciendo cada año en las listas de vulnerabilidades. La buena noticia: usar las herramientas correctas (Uri con parámetros y segmentos, nunca texto pegado) elimina el problema de raíz.

REFACTOR: ordena sin romper nada

Observa que ServicioWikipedia valida en normalizarTema, una función estática pequeña que se prueba sola, sin red. Separar la validación de la petición hace las pruebas más simples.

RETO PARA TI

Agrega una caja de texto a la pantalla para que el usuario escriba el tema (por ejemplo "Guanajuato" o "Talavera"). ¿Qué mensaje ve si escribe 100 letras? (Pista: EntradaInvalidaException tiene un mensaje amable).

INSIGNIA DESBLOQUEADA: Cazador de inyecciones

Ronda 7 · API 5: NASA y el manejo seguro de claves

Esta es la ronda de seguridad. La NASA es la única de las cinco APIs que exige una clave, así que aquí aprenderás cómo se maneja un secreto de verdad.

7.1 · ¿Dónde se filtran las claves?

- En el código fuente: String clave = "abc123"; y luego el código va a GitHub.
- En el repositorio: un archivo .env o secrets subido por accidente.
- Dentro de la app instalada: cualquiera puede descomprimir un .apk (es un .zip) y leer los textos.
- En los logs: print(url) cuando la URL lleva la clave.
- En los mensajes de error que se muestran en pantalla o se mandan a servicios de reportes.
- En capturas de pantalla y chats: "mira, no me funciona" con la clave visible.

> **Dato curioso / ejemplo**
> Los robots que rastrean GitHub buscando patrones de claves (como los de AWS, Google o NASA) encuentran una clave publicada en menos de un minuto. No es paranoia: es automatización. Por eso la regla es "asume que lo que subes, ya lo vio alguien".

7.2 · La escalera de protección

Ninguna medida es perfecta; se combinan en capas, como las capas de una cebolla:

12. Nivel 1 · No escribir la clave en el código: entra al compilar con --dart-define.
13. Nivel 2 · No subirla a git: .gitignore desde el primer commit.
14. Nivel 3 · Mínimo privilegio: usa claves restringidas (solo lectura, solo esa API, con límite de uso) y fáciles de revocar y rotar.
15. Nivel 4 · Proxy en tu propio servidor: para claves valiosas (pagos, IA, mapas con cobro), la app NUNCA tiene la clave; llama a TU servidor y este llama a la API con la clave guardada ahí.
16. Nivel 5 · Datos del usuario (tokens de sesión): se guardan en almacenamiento cifrado del sistema (Keychain en iOS, Keystore en Android) con el paquete flutter_secure_storage, nunca en SharedPreferences.

> **⚠ Cuidado**
> Verdad incómoda: lo que va dentro de la app instalada NO es realmente secreto. --dart-define evita que la clave quede en el repositorio, pero se compila dentro del programa y alguien con herramientas puede encontrarla. Por eso para claves que cuestan dinero se usa el Nivel 4. Para la NASA (gratuita y de bajo riesgo) el Nivel 1 a 3 es razonable, y es perfecta para aprender el hábito.

7.3 · Ciclo ROJO: pruebas de seguridad

Estas pruebas no verifican funciones; verifican que NO se filtren secretos:

test/seguridad_test.dart

```
import 'package:dolores_hoy/datos/config_claves.dart';
import 'package:dolores_hoy/dominio/excepciones.dart';
import 'package:dolores_hoy/dominio/modelos.dart';
import 'package:flutter_test/flutter_test.dart';
```

```
void main() {
  group('ConfigClaves', () {
    test('toString nunca imprime la clave completa', () {
      const c = ConfigClaves(nasa: 'abcdef123456');
      expect(c.toString(), isNot(contains('abcdef123456')));
      expect(c.toString(), contains('ab***56'));
    });

    test('claves muy cortas se ocultan por completo', () {
      expect(const ConfigClaves(nasa: 'abc').toString(), contains('****'));
    });

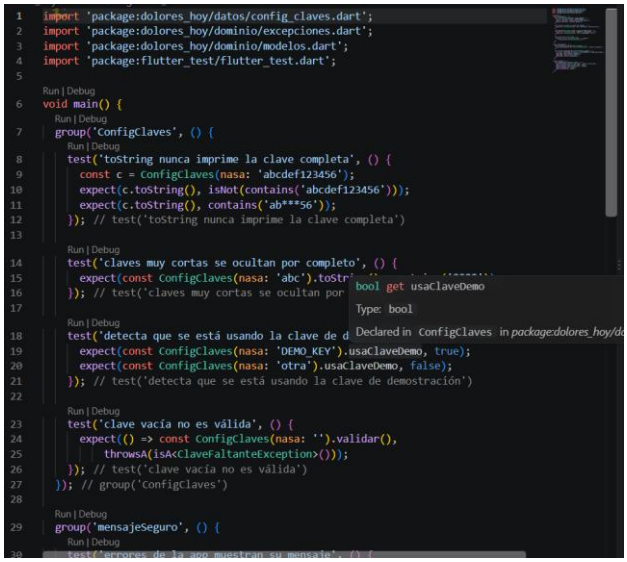
    test('detecta que se está usando la clave de demostración', () {
      expect(const ConfigClaves(nasa: 'DEMO_KEY').usaClaveDemo, true);
      expect(const ConfigClaves(nasa: 'otra').usaClaveDemo, false);
    });

    test('clave vacía no es válida', () {
      expect(() => const ConfigClaves(nasa: '').validar(),
          throwsA(isA<ClaveFaltanteException>()));
    });
  });

  group('mensajeSeguro', () {
    test('errores de la app muestran su mensaje', () {
      expect(mensajeSeguro(const SinConexionException()), contains('conexión'));
    });

    test('errores inesperados NO filtran detalles internos', () {
      final msg = mensajeSeguro(StateError('clave=SECRETO en /home/usuario/app'));
      expect(msg, isNot(contains('SECRETO')));
      expect(msg, isNot(contains('/home')));
      expect(msg, contains('inesperado'));
    });
  });

  group('soloHttps', () {
    test('acepta https y rechaza lo demás', () {
      expect(soloHttps('https://a.com/x.jpg'), 'https://a.com/x.jpg');
      expect(soloHttps('http://a.com/x.jpg'), isNull);
      expect(soloHttps('javascript:alert(1)'), isNull);
      expect(soloHttps(null), isNull);
      expect(soloHttps(42), isNull);
    });
  });
}
```



lib/datos/config_claves.dart

```
import '../dominio/excepciones.dart';

/// Las claves NUNCA se escriben en el código. Entran al compilar:
///   flutter run --dart-define=NASA_API_KEY=tu_clave
///   flutter run --dart-define-from-file=secrets.json
class ConfigClaves {
  final String nasa;
  const ConfigClaves({required this.nasa});

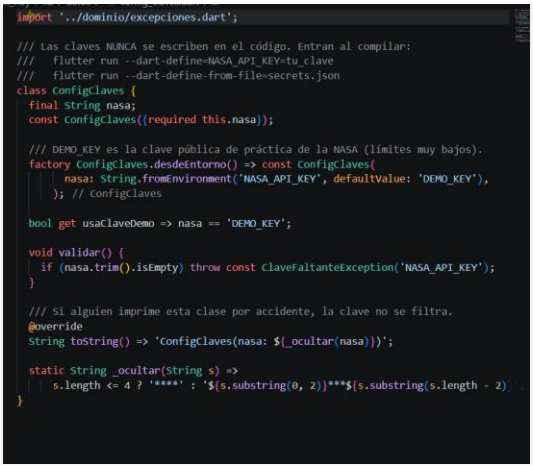
  /// DEMO_KEY es la clave pública de práctica de la NASA (límites muy bajos).
  factory ConfigClaves.desdeEntorno() => const ConfigClaves(
        nasa: String.fromEnvironment('NASA_API_KEY', defaultValue: 'DEMO_KEY'),
      );

  bool get usaClaveDemo => nasa == 'DEMO_KEY';

  void validar() {
    if (nasa.trim().isEmpty) throw const ClaveFaltanteException('NASA_API_KEY');
  }

  /// Si alguien imprime esta clase por accidente, la clave no se filtra.
  @override
  String toString() => 'ConfigClaves(nasa: ${_ocultar(nasa)})';

  static String _ocultar(String s) =>
      s.length <= 4 ? '****' : '${s.substring(0, 2)}***${s.substring(s.length - 2)}';
}
```



```dart
import '../dominio/excepciones.dart';

/// Las claves NUNCA se escriben en el código. Entran al compilar:
///   flutter run --dart-define=NASA_API_KEY=tu_clave
///   flutter run --dart-define-from-file=secrets.json
class ConfigClaves {
  final String nasa;
  const ConfigClaves({required this.nasa});

  /// DEMO_KEY es la clave pública de práctica de la NASA (límites muy bajos).
  factory ConfigClaves.desdeEntorno() => const ConfigClaves(
        nasa: String.fromEnvironment('NASA_API_KEY', defaultValue: 'DEMO_KEY'),
      ); // ConfigClaves

  bool get usaClaveDemo => nasa == 'DEMO_KEY';

  void validar() {
    if (nasa.trim().isEmpty) throw const ClaveFaltanteException('NASA_API_KEY');
  }

  /// Si alguien imprime esta clase por accidente, la clave no se filtra.
  @override
  String toString() => 'ConfigClaves(nasa: ${_ocultar(nasa)})';

  static String _ocultar(String s) =>
      s.length <= 4 ? '****' : '${s.substring(0, 2)}***${s.substring(s.length - 2)}';
}
```

Lectura guiada

17. String.fromEnvironment lee el valor que se pasó al compilar con --dart-define. Es una constante de compilación: no se lee de ningún archivo en tiempo de ejecución.
18. defaultValue: DEMO_KEY permite que los alumnos practiquen sin registrarse, pero usaClaveDemo avisa que los límites son bajos.
19. validar() lanza ClaveFaltanteException antes de hacer la petición: no se gasta una llamada de red con una clave vacía.
20. toString() oculta la clave: si alguien imprime el objeto por accidente (print(claves)), solo verá "ab***56".

7.5 · Ciclo ROJO y VERDE del servicio NASA

test/servicios_test.dart (grupo 5)

```dart
group('API 5 · NASA APOD (con clave)', () {
  test('manda la clave como parámetro api_key', () async {
    final s = ServicioNasa(
        seguro((req) async {
          expect(req.url.host, 'api.nasa.gov');
          expect(req.url.queryParameters['api_key'], 'MI_CLAVE');
          return jsonOk(jsonNasa);
        }),
        const ConfigClaves(nasa: 'MI_CLAVE'));
    final f = await s.fotoDelDia();
    expect(f.titulo, 'Nebulosa de ejemplo');
    expect(f.urlImagen, isNotNull);
  });

  test('sin clave ni siquiera hace la petición', () {
    var llamado = false;
    final s = ServicioNasa(
        seguro((_) async {
          llamado = true;
          return jsonOk(jsonNasa);
        }),
        const ConfigClaves(nasa: '  '));
    expect(s.fotoDelDia(), throwsA(isA<ClaveFaltanteException>()));
    expect(llamado, false);
  });
```

```
    test('si la NASA publica un video, no hay imagen que mostrar', () async {
      final s = ServicioNasa(
          seguro((_) async => jsonOk({...jsonNasa, 'media_type': 'video', 'url':
'https://youtube.com/x'})),
          const ConfigClaves(nasa: 'K'));
      expect((await s.fotoDelDia()).urlImagen, isNull);
    });

    test('una URL de imagen http:// se descarta', () async {
      final s = ServicioNasa(
          seguro((_) async => jsonOk({...jsonNasa, 'url': 'http://apod.nasa.gov/a.jpg'})),
          const ConfigClaves(nasa: 'K'));
      expect((await s.fotoDelDia()).urlImagen, isNull);
    });

    test('403 explica que la clave no es válida', () {
      final s = ServicioNasa(
          seguro((_) async => jsonOk({}, 403)), const ConfigClaves(nasa: 'MALA'));
      expect(s.fotoDelDia(),
          throwsA(isA<ErrorServidorException>().having((e) => e.toString(), 'm',
contains('clave'))));
    });
  });
```



```
group('API 5 · NASA APOD (con clave)', () {
  test('manda la clave como parámetro api_key', () async {
    final s = ServicioNasa(
        seguro((req) async {
          expect(req.url.host, 'api.nasa.gov');
          expect(req.url.queryParameters['api_key'], 'MI_CLAVE');
          return jsonOk(jsonNasa);
        }),
        const ConfigClaves(nasa: 'MI_CLAVE')); // ServicioNasa
    final f = await s.fotoDelDia();
    expect(f.titulo, 'Nebulosa de ejemplo');
    expect(f.urlImagen, isNotNull);
  }); // test('manda la clave como parámetro api_key')

  test('sin clave ni siquiera hace la petición', () {
    var llamado = false;
    final s = ServicioNasa(
        seguro((_) async {
          llamado = true;
          return jsonOk(jsonNasa);
        }),
        const ConfigClaves(nasa: '')); // ServicioNasa
    expect(s.fotoDelDia(), throwsA(isA<ClaveFaltanteException>()));
    expect(llamado, false);
  }); // test('sin clave ni siquiera hace la petición')

  test('si la NASA publica un video, no hay imagen que mostrar', () async {
    final s = ServicioNasa(
        seguro((_) async => jsonOk({...jsonNasa, 'media_type': 'video', 'url': '
        const ConfigClaves(nasa: 'K')); // ServicioNasa
    expect((await s.fotoDelDia()).urlImagen, isNull);
```

lib/dominio/modelos.dart (agrega)

```
class FotoAstronomica {
  final String titulo;
  final String explicacion;
  final DateTime fecha;
  final String? urlImagen; // null si es video o la URL no es https

  const FotoAstronomica(this.titulo, this.explicacion, this.fecha, this.urlImagen);

  factory FotoAstronomica.desdeJson(Map<String, dynamic> json) => FotoAstronomica(
        json['title'] as String,
        json['explanation'] as String,
        DateTime.parse(json['date'] as String),
        json['media_type'] == 'image' ? soloHttps(json['url']) : null,
      );
}
```

```dart
class FotoAstronomica {
  final String titulo;
  final String explicacion;
  final DateTime fecha;
  final String? urlImagen; // null si es video o la URL no es https

  const FotoAstronomica(this.titulo, this.explicacion, this.fecha, this.urlImagen);

  factory FotoAstronomica.desdeJson(Map<String, dynamic> json) => FotoAstronomica(
        json['title'] as String,
        json['explanation'] as String,
        DateTime.parse(json['date'] as String),
        json['media_type'] == 'image' ? soloHttps(json['url']) : null,
      );
}
```

lib/datos/servicios.dart (agrega)

```dart
/// API 5 · NASA APOD (foto astronómica del día, CON clave)
class ServicioNasa {
  final ClienteSeguro _c;
  final ConfigClaves _claves;
  ServicioNasa(this._c, this._claves);

  Future<FotoAstronomica> fotoDelDia() async {
    _claves.validar();
    final json = await _c.obtenerJson(Uri.https('api.nasa.gov', '/planetary/apod', {
      'api_key': _claves.nasa,
    }));
    return convertir(() => FotoAstronomica.desdeJson(json as Map<String, dynamic>));
  }
}
```


```dart
/// API 5 · NASA APOD (foto astronómica del día, CON clave)
class ServicioNasa {
  final ClienteSeguro _c;
  final ConfigClaves _claves;
  ServicioNasa(this._c, this._claves);

  Future<FotoAstronomica> fotoDelDia() async {
    _claves.validar();
    final json = await _c.obtenerJson(Uri.https('api.nasa.gov', '/planetary/apod', {
      'api_key': _claves.nasa,
    }));
    return convertir(() => FotoAstronomica.desdeJson(json as Map<String, dynamic>));
  }
}
```


Dos defensas de salida que se ven poco

- media_type: la NASA a veces publica un VIDEO en lugar de imagen. Si intentas mostrarlo con Image.network, falla. Solo aceptamos la URL cuando media_type es "image".
- soloHttps(): aunque la NASA es confiable, no confiamos ciegamente. Si una URL de imagen llegara con http:// o con algo raro como "javascript:", se descarta. Validar la salida es tan importante como validar la entrada.

7.6 · Cómo ejecutar con tu clave

```
$ # Opción A: archivo (recomendada)
$ flutter run --dart-define-from-file=secrets.json
$
$ # Opción B: directo
$ flutter run --dart-define=NASA_API_KEY=tu_clave_aqui
$
$ # Para compilar la versión final ofuscando el código:
$ flutter build apk --obfuscate --split-debug-info=build/simbolos --dart-define-from-
file=secrets.json
```

Dato curioso / ejemplo

La ofuscación (--obfuscate) cambia los nombres de clases y funciones por letras sin sentido: no es cifrado, pero le hace la vida más difícil a quien intente leer tu programa. Guarda la carpeta de símbolos: la necesitarás para entender los reportes de errores.

7.7 · Auditoría: ¿qué tan visible es mi clave?

La mejor forma de entender el riesgo es revisar tu propia app. Compila una versión de prueba con una clave falsa que reconozcas y busca ese texto en el resultado (esto lo haces únicamente con TU app y TUS claves de práctica):

```
$ flutter build apk --dart-define=NASA_API_KEY=CLAVE_DE_PRUEBA_12345
$ cd build/app/outputs/flutter-apk
$ unzip -o app-release.apk -d contenido
$ grep -r "CLAVE_DE_PRUEBA_12345" contenido | head
```

Discute en clase qué encontraste o no encontraste (el texto puede aparecer o no según el formato de compilación) y por qué, a pesar de eso, el Nivel 4 (proxy) es la recomendación para cualquier clave que cueste dinero.

7.8 · Límites de uso (rate limiting)

Las APIs gratuitas limitan cuántas peticiones puedes hacer. Si superas el límite, responden 429. Buenas prácticas:

- Mostrar un mensaje claro (ya lo hace ErrorServidorException).
- No reintentar en un ciclo infinito: espera y duplica el tiempo entre reintentos (backoff exponencial).
- Guardar la respuesta un rato (caché): la foto del día de la NASA no cambia en 24 horas; pedirla cada vez que abres la app es un desperdicio y un riesgo de llegar al límite.
- Si te bloquean, no cambies de clave "para evadir" el límite: es contra los términos de uso.

7.9 · Mapa con el OWASP Mobile Top 10

OWASP es una organización que publica las 10 vulnerabilidades más frecuentes en apps móviles. Lo que aplicaste hoy se relaciona con:

- M1 · Uso inadecuado de credenciales → claves fuera del código y del repositorio, ConfigClaves, redactar.
- M4 · Validación insuficiente de entradas y salidas → normalizarTema, rangos de coordenadas y años, soloHttps, media_type.
- M5 · Comunicación insegura → https obligatorio en ClienteSeguro.
- M8 · Configuración de seguridad incorrecta → no activar cleartext, permisos mínimos.
- M9 · Almacenamiento inseguro de datos → (próximo paso) flutter_secure_storage para tokens de usuario.

RETO PARA TI

> Cambia temporalmente la dirección de Open-Meteo a http:// en el servicio. ¿Qué prueba falla y con qué mensaje? Vuelve a dejarla en https.

INSIGNIA DESBLOQUEADA: Guardián de secretos

Ronda 8 · El tablero: 5 APIs, cero dependencias entre ellas

El principio de la "puerta de emergencia"

Si en un cine se incendia una sala, las otras cuatro no se evacuan. Cada API es una sala independiente: si la NASA está caída, el clima, el tipo de cambio, los festivos y Wikipedia deben seguir funcionando. Esto se llama resiliencia o "degradación elegante".

Para lograrlo, cada API tiene su propio estado (Carga) y el ViewModel atrapa los errores UNO POR UNO. Además las cinco peticiones salen al mismo tiempo con Future.wait, así que el tablero tarda lo que tarda la más lenta, no la suma de las cinco.

> **Dato curioso / ejemplo**
> Si cada API tarda 1 segundo: en serie son 5 segundos; en paralelo, 1 segundo. Es como cinco cajeros atendiendo a la vez en lugar de una sola fila.

Ciclo 8.1 · ROJO

test/tablero_viewmodel_test.dart

```
import 'package:dolores_hoy/datos/config_claves.dart';
import 'package:dolores_hoy/datos/servicios.dart';
import 'package:dolores_hoy/presentacion/tablero_viewmodel.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:http/http.dart' as http;

import 'ayudas.dart';

TableroViewModel crear(Future<http.Response> Function(http.Request) h) {
  final c = seguro(h);
  return TableroViewModel(
    clima: ServicioClima(c),
    cambio: ServicioCambio(c),
    festivos: ServicioFestivos(c),
    wikipedia: ServicioWikipedia(c),
    nasa: ServicioNasa(c, const ConfigClaves(nasa: 'DEMO_KEY')),
  );
}

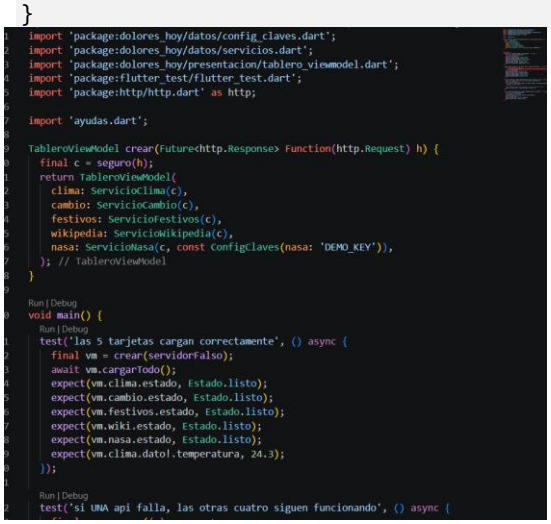
void main() {
  test('las 5 tarjetas cargan correctamente', () async {
    final vm = crear(servidorFalso);
    await vm.cargarTodo();
    expect(vm.clima.estado, Estado.listo);
    expect(vm.cambio.estado, Estado.listo);
    expect(vm.festivos.estado, Estado.listo);
    expect(vm.wiki.estado, Estado.listo);
    expect(vm.nasa.estado, Estado.listo);
    expect(vm.clima.dato!.temperatura, 24.3);
  });

  test('si UNA api falla, las otras cuatro siguen funcionando', () async {
    final vm = crear((r) async =>
        r.url.host == 'api.nasa.gov' ? jsonOk({}, 500) : servidorFalso(r));
    await vm.cargarTodo();
    expect(vm.nasa.estado, Estado.error);
    expect(vm.nasa.error, contains('500'));
    expect(vm.clima.estado, Estado.listo);
```

```
    expect(vm.cambio.estado, Estado.listo);
    expect(vm.wiki.estado, Estado.listo);
  });

  test('sin internet, todas muestran error pero la app no se cae', () async {
    final vm = crear((_) async => throw Exception('sin red'));
    await vm.cargarTodo();
    for (final e in [vm.clima.estado, vm.cambio.estado, vm.festivos.estado, vm.wiki.estado,
vm.nasa.estado]) {
      expect(e, Estado.error);
    }
    // El mensaje es genérico: no revela el texto interno del error.
    expect(vm.clima.error, isNot(contains('sin red')));
  });

  test('cargarTodo anuncia el estado cargando antes de terminar', () async {
    final vm = crear(servidorFalso);
    final vistos = <Estado>[];
    vm.addListener(() => vistos.add(vm.clima.estado));
    await vm.cargarTodo();
    expect(vistos.first, Estado.cargando);
    expect(vistos.last, Estado.listo);
  });
}
```



```
import 'package:dolores_hoy/datos/config_claves.dart';
import 'package:dolores_hoy/datos/servicios.dart';
import 'package:dolores_hoy/presentacion/tablero_viewmodel.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:http/http.dart' as http;

import 'ayudas.dart';

TableroViewModel crear(Future<http.Response> Function(http.Request) h) {
  final c = seguro(h);
  return TableroViewModel(
    clima: ServicioClima(c),
    cambio: ServicioCambio(c),
    festivos: ServicioFestivos(c),
    wikipedia: ServicioWikipedia(c),
    nasa: ServicioNasa(c, const ConfigClaves(nasa: 'DEMO_KEY')),
  ); // TableroViewModel
}

Run | Debug
void main() {
  Run | Debug
  test('las 5 tarjetas cargan correctamente', () async {
    final vm = crear(servidorFalso);
    await vm.cargarTodo();
    expect(vm.clima.estado, Estado.listo);
    expect(vm.cambio.estado, Estado.listo);
    expect(vm.festivos.estado, Estado.listo);
    expect(vm.wiki.estado, Estado.listo);
    expect(vm.nasa.estado, Estado.listo);
    expect(vm.clima.dato!.temperatura, 24.3);
  });

  Run | Debug
  test('si UNA api falla, las otras cuatro siguen funcionando', () async {
```

Mira la prueba "si UNA api falla": el servidor falso devuelve 500 solo para la NASA y verificamos que las demás tarjetas estén en listo. Y la de "sin internet": el mensaje mostrado NO contiene el texto interno del error.

Ciclo 8.2 · VERDE

lib/presentacion/tablero_viewmodel.dart

```
import 'package:flutter/foundation.dart';

import '../datos/servicios.dart';
import '../dominio/excepciones.dart';
import '../dominio/modelos.dart';

enum Estado { cargando, listo, error }

/// Estado de UNA tarjeta. Cada API tiene el suyo: si una falla, las demás viven.
class Carga<T> {
```

```
  final Estado estado;
  final T? dato;
  final String? error;
  const Carga._(this.estado, this.dato, this.error);
  const Carga.cargando() : this._(Estado.cargando, null, null);
  const Carga.listo(T dato) : this._(Estado.listo, dato, null);
  const Carga.fallo(String error) : this._(Estado.error, null, error);
}

class TableroViewModel extends ChangeNotifier {
  final ServicioClima _clima;
  final ServicioCambio _cambio;
  final ServicioFestivos _festivos;
  final ServicioWikipedia _wikipedia;
  final ServicioNasa _nasa;

  TableroViewModel({
    required ServicioClima clima,
    required ServicioCambio cambio,
    required ServicioFestivos festivos,
    required ServicioWikipedia wikipedia,
    required ServicioNasa nasa,
  })  : _clima = clima,
        _cambio = cambio,
        _festivos = festivos,
        _wikipedia = wikipedia,
        _nasa = nasa;

  Carga<Clima> clima = const Carga.cargando();
  Carga<TipoCambio> cambio = const Carga.cargando();
  Carga<List<Festivo>> festivos = const Carga.cargando();
  Carga<ResumenWiki> wiki = const Carga.cargando();
  Carga<FotoAstronomica> nasa = const Carga.cargando();

  Future<void> cargarTodo() async {
    clima = const Carga.cargando();
    cambio = const Carga.cargando();
    festivos = const Carga.cargando();
    wiki = const Carga.cargando();
    nasa = const Carga.cargando();
    notifyListeners();

    // Las 5 peticiones salen al mismo tiempo (en paralelo).
    await Future.wait([
      _cargar(() => _clima.actual(), (c) => clima = c),
      _cargar(() => _cambio.deMxn(), (c) => cambio = c),
      _cargar(() => _festivos.proximos(), (c) => festivos = c),
      _cargar(() => _wikipedia.resumen('Dolores_Hidalgo'), (c) => wiki = c),
      _cargar(() => _nasa.fotoDelDia(), (c) => nasa = c),
    ]);
  }

  Future<void> _cargar<T>(Future<T> Function() pedir, void Function(Carga<T>) poner) async {
    try {
      poner(Carga.listo(await pedir()));
    } catch (e) {
      poner(Carga.fallo(mensajeSeguro(e)));
    }
    notifyListeners();
  }
}
```

```
$ flutter test
```

```dart
import 'package:flutter/foundation.dart';

import '../datos/servicios.dart';
import '../dominio/excepciones.dart';
import '../dominio/modelos.dart';

enum Estado { cargando, listo, error }

/// Estado de UNA tarjeta. Cada API tiene el suyo: si una falla, las demás viver
class Carga<T> {
  final Estado estado;
  final T? dato;
  final String? error;
  const Carga._(this.estado, this.dato, this.error);
  const Carga.cargando() : this._(Estado.cargando, null, null);
  const Carga.listo(T dato) : this._(Estado.listo, dato, null);
  const Carga.fallo(String error) : this._(Estado.error, null, error);
}

class TableroViewModel extends ChangeNotifier {
  final ServicioClima _clima;
  final ServicioCambio _cambio;
  final ServicioFestivos _festivos;
  final ServicioWikipedia _wikipedia;
  final ServicioNasa _nasa;

  TableroViewModel({
    required ServicioClima clima,
    required ServicioCambio cambio,
    required ServicioFestivos festivos,
    required ServicioWikipedia wikipedia,
    required ServicioNasa nasa,
  })  : _clima = clima,
        _cambio = cambio,
        _festivos = festivos,
        _wikipedia = wikipedia,
```

REFACTOR: ordena sin romper nada

Ejecuta flutter test: deberías ver pasar todas las pruebas (cliente, 5 servicios, seguridad y tablero) sin internet y en pocos segundos. Compara: probar contra las APIs reales sería lento, inestable y gastaría tu cupo.

⚠ Cuidado

En _cargar() el catch atrapa TODO tipo de error, pero lo pasa por mensajeSeguro(). Atrapar todo evita que un bug tumbe la pantalla; pasar por mensajeSeguro evita que el bug filtre información. En un proyecto real, además registrarías el error completo en un servicio de reportes (sin secretos).

INSIGNIA DESBLOQUEADA: Director de orquesta

Ronda 9 · La pantalla y la prueba final

Paso 9.1 · La pantalla

La pantalla es "tonta" a propósito: solo dibuja lo que dice el ViewModel. TarjetaCarga<T> es un widget genérico que sirve para las cinco APIs: muestra barra de carga, mensaje de error o el contenido según el estado.

lib/presentacion/pantalla_tablero.dart

```
import 'package:flutter/material.dart';

import '../dominio/modelos.dart';
import 'tablero_viewmodel.dart';

class PantallaTablero extends StatefulWidget {
  final TableroViewModel vm;
  const PantallaTablero({super.key, required this.vm});
  @override
  State<PantallaTablero> createState() => _PantallaTableroState();
}

class _PantallaTableroState extends State<PantallaTablero> {
  @override
  void initState() {
    super.initState();
    widget.vm.cargarTodo();
  }

  @override
  Widget build(BuildContext context) {
    final vm = widget.vm;
    return AnimatedBuilder(
      animation: vm,
      builder: (_, __) => Scaffold(
        appBar: AppBar(title: const Text('Dolores Hoy')),
        body: RefreshIndicator(
          onRefresh: vm.cargarTodo,
          child: ListView(padding: const EdgeInsets.all(12), children: [
            TarjetaCarga<Clima>(
              titulo: 'Clima en Dolores Hidalgo', icono: Icons.wb_sunny, carga: vm.clima,
              construir: (c) {
                final (emoji, texto) = describirClima(c.codigo);
                return Text('$emoji $texto · ${c.temperatura.toStringAsFixed(1)} °C\n'
                    'Humedad ${c.humedad}% · Viento ${c.viento.toStringAsFixed(0)} km/h');
              },
            ),
            TarjetaCarga<TipoCambio>(
              titulo: 'Tipo de cambio', icono: Icons.attach_money, carga: vm.cambio,
              construir: (t) => Text(
                  '1 USD = \$${t.pesosPorUnidad('USD').toStringAsFixed(2)} MXN\n'
                  '1 EUR = \$${t.pesosPorUnidad('EUR').toStringAsFixed(2)} MXN\n'
                  'Actualizado: ${t.fecha.toIso8601String().substring(0, 10)}'),
            ),
            TarjetaCarga<List<Festivo>>(
              titulo: 'Próximos festivos', icono: Icons.celebration, carga: vm.festivos,
              construir: (lista) => Text(lista.isEmpty
                  ? 'No hay más festivos este año.'
                  : lista.map((f) =>
'${f.fecha.day}/${f.fecha.month} · ${f.nombre}').join('\n')),
            ),
```

```
            TarjetaCarga<ResumenWiki>(
              titulo: 'Wikipedia', icono: Icons.menu_book, carga: vm.wiki,
              construir: (w) => Column(crossAxisAlignment: CrossAxisAlignment.start, children:
[
                Text(w.titulo, style: Theme.of(context).textTheme.titleMedium),
                const SizedBox(height: 4),
                Text(w.extracto, maxLines: 6, overflow: TextOverflow.ellipsis),
              ]),
            ),
            TarjetaCarga<FotoAstronomica>(
              titulo: 'Foto astronómica (NASA)', icono: Icons.rocket_launch, carga: vm.nasa,
              construir: (f) => Column(crossAxisAlignment: CrossAxisAlignment.start, children:
[
                Text(f.titulo, style: Theme.of(context).textTheme.titleMedium),
                const SizedBox(height: 8),
                if (f.urlImagen != null)
                  Image.network(f.urlImagen!,
                      errorBuilder: (_, __, ___) => const Text('No se pudo cargar la imagen'))
                else
                  const Text('(Hoy la NASA publicó un video; no se muestra por seguridad)'),
              ]),
            ),
          ]),
        ),
      ),
    );
  }
}

class TarjetaCarga<T> extends StatelessWidget {
  final String titulo;
  final IconData icono;
  final Carga<T> carga;
  final Widget Function(T) construir;

  const TarjetaCarga({
    super.key,
    required this.titulo,
    required this.icono,
    required this.carga,
    required this.construir,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(14),
        child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
          Row(children: [
            Icon(icono),
            const SizedBox(width: 8),
            Text(titulo, style: Theme.of(context).textTheme.titleSmall),
          ]),
          const SizedBox(height: 10),
          switch (carga.estado) {
            Estado.cargando => const LinearProgressIndicator(),
            Estado.error => Text(carga.error ?? 'Error',
                style: TextStyle(color: Theme.of(context).colorScheme.error)),
            Estado.listo => construir(carga.dato as T),
          },
        ]),
      ),
```

```
    );
  }
}
```



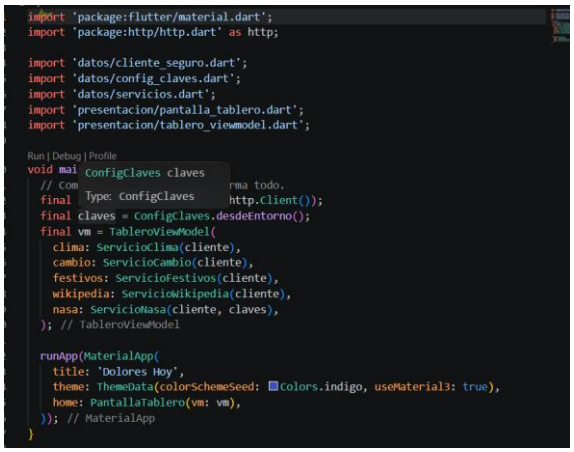
Paso 9.2 · main.dart (el único lugar donde se arma todo)

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

import 'datos/cliente_seguro.dart';
import 'datos/config_claves.dart';
import 'datos/servicios.dart';
import 'presentacion/pantalla_tablero.dart';
import 'presentacion/tablero_viewmodel.dart';

void main() {
  // Composition root: aquí se arma todo.
  final cliente = ClienteSeguro(http.Client());
  final claves = ConfigClaves.desdeEntorno();
  final vm = TableroViewModel(
    clima: ServicioClima(cliente),
    cambio: ServicioCambio(cliente),
    festivos: ServicioFestivos(cliente),
    wikipedia: ServicioWikipedia(cliente),
    nasa: ServicioNasa(cliente, claves),
  );
  runApp(MaterialApp(
    title: 'Dolores Hoy',
    theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true),
    home: PantallaTablero(vm: vm),
  ));
}
```



Paso 9.3 · Prueba de fuego

21. Ejecuta: flutter run --dart-define-from-file=secrets.json
22. Deberías ver las 5 tarjetas llenarse casi al mismo tiempo.
23. Jala hacia abajo para recargar (RefreshIndicator).
24. Activa el modo avión y recarga: las 5 tarjetas muestran un mensaje amable, la app NO se cierra.
25. Desactívalo y recarga: todo regresa.
26. Prueba con una clave falsa de la NASA: solo esa tarjeta muestra el error de clave inválida (403); las otras cuatro siguen bien.

> **Dato curioso / ejemplo**
> El punto 5 es la demostración más bonita de resiliencia: rompes una API a propósito y el resto de la app ni se entera. Eso es lo que hace "profesional" a una aplicación.

INSIGNIA DESBLOQUEADA: Integrador seguro

Cierre · Lo que lograste

Checklist de seguridad (revisa antes de entregar)

- Ninguna clave está escrita en el código ni en el repositorio (git status limpio, secrets.json ignorado).
- Todas las peticiones usan https y el cliente rechaza http.
- Toda petición tiene timeout y límite de tamaño de respuesta.
- Las direcciones se construyen con Uri.https / pathSegments, no con texto pegado.
- Toda entrada del usuario se valida (vacío, longitud, caracteres de control).
- Toda salida del servidor se valida (tipos, media_type, URLs https).
- Los logs usan redactar(); ConfigClaves no imprime la clave completa.
- Los errores inesperados muestran un mensaje genérico al usuario.
- Una API caída no afecta a las demás.
- La clave tiene el menor privilegio posible y se puede revocar.

Preguntas para discutir en clase

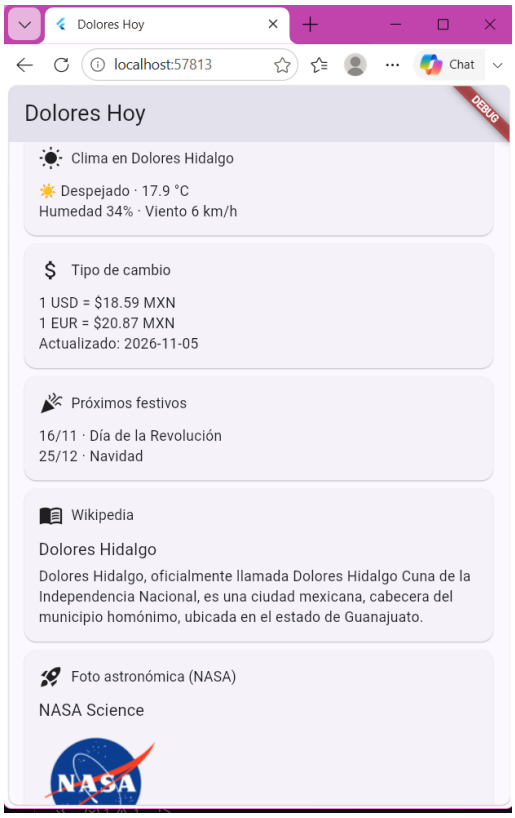
27. ¿Por qué --dart-define no es suficiente protección para una clave que cuesta dinero? ¿Qué harías en su lugar?
28. Un compañero sube su secrets.json a GitHub. ¿Qué pasos sigue, en orden, para reparar el problema?
29. ¿Qué ventaja tiene que TODAS las APIs pasen por una sola clase (ClienteSeguro)?
30. ¿Por qué validamos la respuesta del servidor si "la NASA es confiable"?
31. Si la API de Wikipedia cambia el nombre del campo "extract", ¿qué ve el usuario? ¿Y qué prueba te avisaría primero?

Retos finales por equipos

- Caché de 1 hora: guarda la última respuesta de cada API en memoria y úsala si la petición falla (modo sin conexión).
- Reintento con backoff exponencial para errores 429 y de red (1 s, 2 s, 4 s) con pruebas primero.
- Servidor proxy: reutiliza la API local en Node de la práctica anterior para que la clave de la NASA viva en el servidor y la app ya no la contenga.
- Guarda el último tema buscado con flutter_secure_storage y escribe la prueba con un doble.
- Agrega una sexta API gratuita de tu elección (por ejemplo, PokeAPI o Open Library) aplicando TODO el checklist.

🏅 INSIGNIA DESBLOQUEADA: Desarrollador de apps conectadas y seguras

Recuerda: la seguridad no es un archivo que se agrega al final; es un hábito en cada línea. ¡Éxito en tu tablero!


Dolores Hoy
localhost:57813
Chat
Dolores Hoy
DEBUG
Clima en Dolores Hidalgo
Despejado · 17.9 °C
Humedad 34% · Viento 6 km/h
Tipo de cambio
1 USD = $18.59 MXN
1 EUR = $20.87 MXN
Actualizado: 2026-11-05
Próximos festivos
16/11 · Día de la Revolución
25/12 · Navidad
Wikipedia
Dolores Hidalgo
Dolores Hidalgo, oficialmente llamada Dolores Hidalgo Cuna de la Independencia Nacional, es una ciudad mexicana, cabecera del municipio homónimo, ubicada en el estado de Guanajuato.
Foto astronómica (NASA)
NASA Science
NASA